

LINEAR REGRESSION USING A MULTI-LAYER NEURAL NETWORK FROM SCRATCH

Ankit Saini (2522MA01)
Tiyas Chakarbortty (2522MA02)
Tanu Goliyan (2621MA05)
Murari Ray (2621MA07)

Department of Mathematics
Indian Institute of Technology Patna

1. Introduction

This work develops a small multi-layer neural network for a regression problem. The aim is not to use a ready-made neural-network model, but to implement the main steps directly so that the learning process can be followed clearly.

The California Housing dataset is used to predict the median house value (`MedHouseVal`).

Only two variables, `MedInc` and `HouseAge`, are used as input features. Since the target is numerical, the task is treated as a regression problem even though the predictor itself is a multi-layer neural network.

The network contains two hidden layers. The first hidden layer has five neurons, the second has three neurons, and ReLU is used in both hidden layers. The final layer has one linear output neuron.

Two learning rates are tested with both Gradient Descent and Momentum.

2. Objectives

The objectives of this work are:

1. Build a neural network for regression without using a pre-built neural-network model.
2. Predict `MedHouseVal` using `MedInc` and `HouseAge`.
3. Implement forward propagation and backpropagation.
4. Use Mean Squared Error (MSE) as the loss function.

5. Study learning rates 0.01 and 0.001.
6. Compare Gradient Descent and Momentum.
7. Observe the training loss and convergence behaviour.

3. Dataset and Preprocessing

The notebook loads the California Housing dataset using scikit-learn. The complete dataset contains 20,640 samples and 8 original predictor variables plus the target.

For this experiment, the input matrix is reduced to the two columns `MedInc` and `HouseAge`, while `MedHouseVal` is used as the target.

The data is randomly shuffled with seed 42 and divided into 80% training and 20% testing.

This gives 16,512 training observations and 4,128 testing observations.

The mean and standard deviation are calculated only from the training data and then used to normalize both training and testing inputs.

4. Methodology

4.1 Network Structure

The implemented architecture is

$$2 \longrightarrow 5 \longrightarrow 3 \longrightarrow 1$$

Table 1: Network Architecture

Layer	Neurons	Activation
Input layer	2	—
Hidden layer 1	5	ReLU
Hidden layer 2	3	ReLU
Output layer	1	Linear

4.2 Forward Propagation

For each sample, the normalized inputs are passed through the first hidden layer, then the second hidden layer, and finally the linear output layer.

The calculations are:

$$Z_1 = XW_1 + b_1$$

$$A_1 = \text{ReLU}(Z_1)$$

$$Z_2 = A_1 W_2 + b_2$$

$$A_2 = \text{ReLU}(Z_2)$$

$$\hat{y} = A_2 W_3 + b_3$$

4.3 ReLU Activation

ReLU is used in both hidden layers.

It keeps positive values and changes negative values to zero:

$$\text{ReLU}(z) = \max(0, z)$$

The derivative of ReLU is used during backpropagation.

4.4 Loss Function

Mean Squared Error measures the difference between the actual target and the network prediction:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

The objective of training is to reduce this loss by repeatedly updating the weights and biases.

4.5 Backpropagation

Backpropagation starts at the output layer and works backward through the second and first hidden layers.

The gradients of all weights and biases are calculated using the chain rule. These gradients are then passed to the selected optimization method.

4.6 Optimization Methods

Gradient Descent:

Each parameter is moved in the opposite direction of its gradient using the selected learning rate.

Momentum:

An exponentially weighted velocity is maintained for each parameter. $\beta = 0.9$ is used, allowing previous updates to influence the current update.

5. Input parameters and their shapes

Table 2: Parameter Dimensions

Parameter	Shape
W_1	(2, 5)
b_1	(1, 5)
W_2	(5, 3)
b_2	(1, 3)
W_3	(3, 1)
b_3	(1, 1)

The initial prediction made by the randomly initialized network gives an MSE of approximately

5.60346

before training.

5.1 Training Configuration

Table 3: Experimental Configuration

Item	Value
Dataset	California Housing
Input features	MedInc, HouseAge
Target	MedHouseVal
Training / testing split	80% / 20%
Architecture	$2 \rightarrow 5 \rightarrow 3 \rightarrow 1$
Hidden activation	ReLU
Output activation	Linear
Loss	Mean Squared Error
Learning rates	0.01 and 0.001
Epochs	1000
Momentum β	0.9

6. Experimental Results

Four training runs were completed.

Gradient Descent and Momentum were each tested at learning rates 0.01 and 0.001 for 1000 epochs.

Table 4: Comparison of Training Runs

Optimizer	Learning Rate	Final Training MSE	Time (s)
Gradient Descent	0.01	1.327942	6.079501
Gradient Descent	0.001	1.406153	7.208127
Momentum	0.01	1.327937	4.753206
Momentum	0.001	1.402953	3.767043

Best training result:

Momentum with learning rate 0.01 gives the smallest final training MSE approximately 1.327937.

The Gradient Descent run at 0.01 is almost identical.

6.1 Gradient Descent

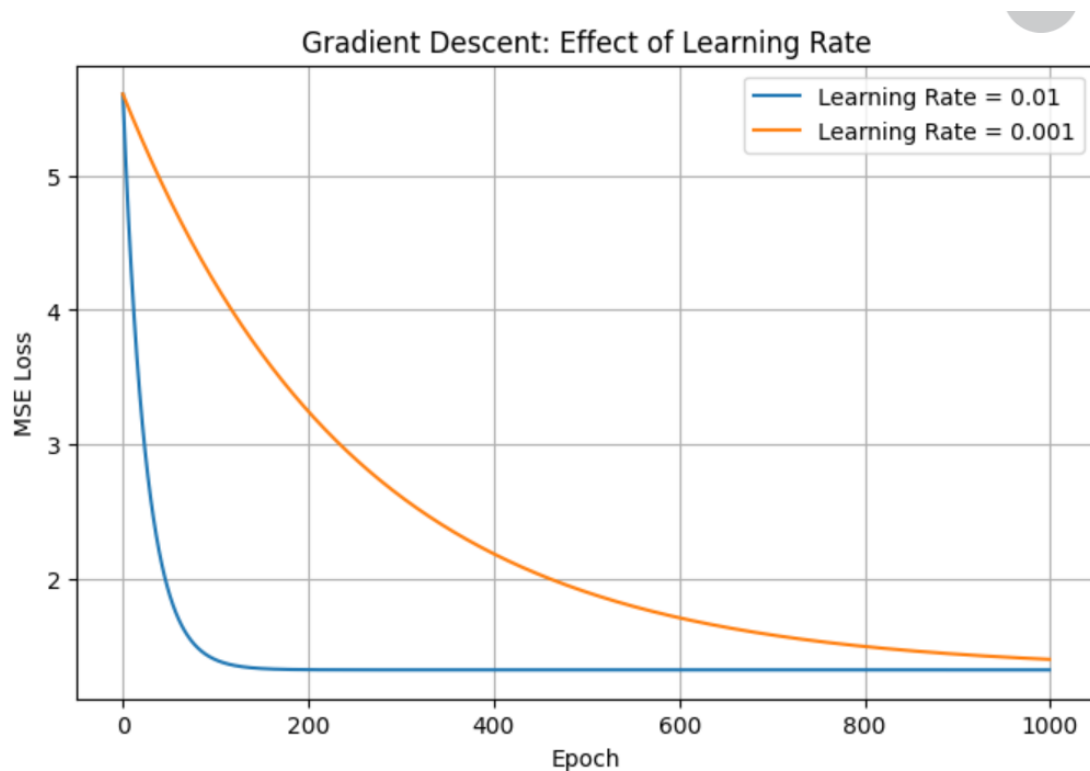


Figure 1: Gradient Descent comparison for learning rates 0.01 and 0.001.

6.2 Momentum

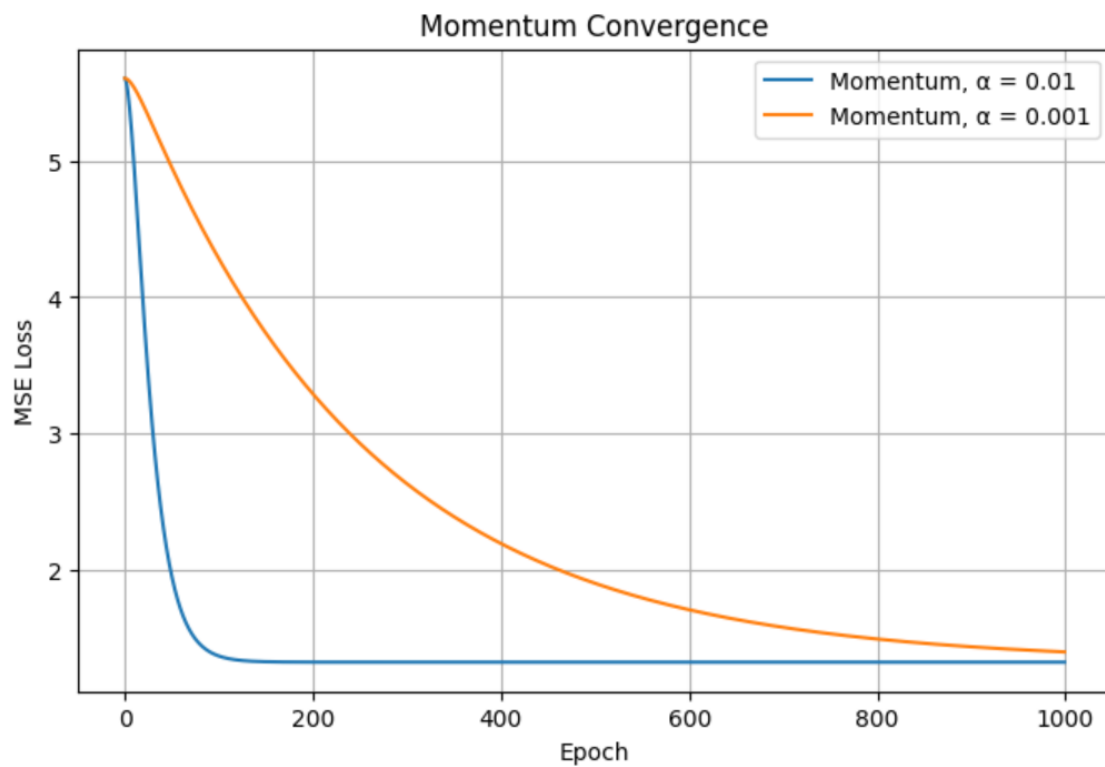


Figure 2: Momentum convergence for the two learning rates.

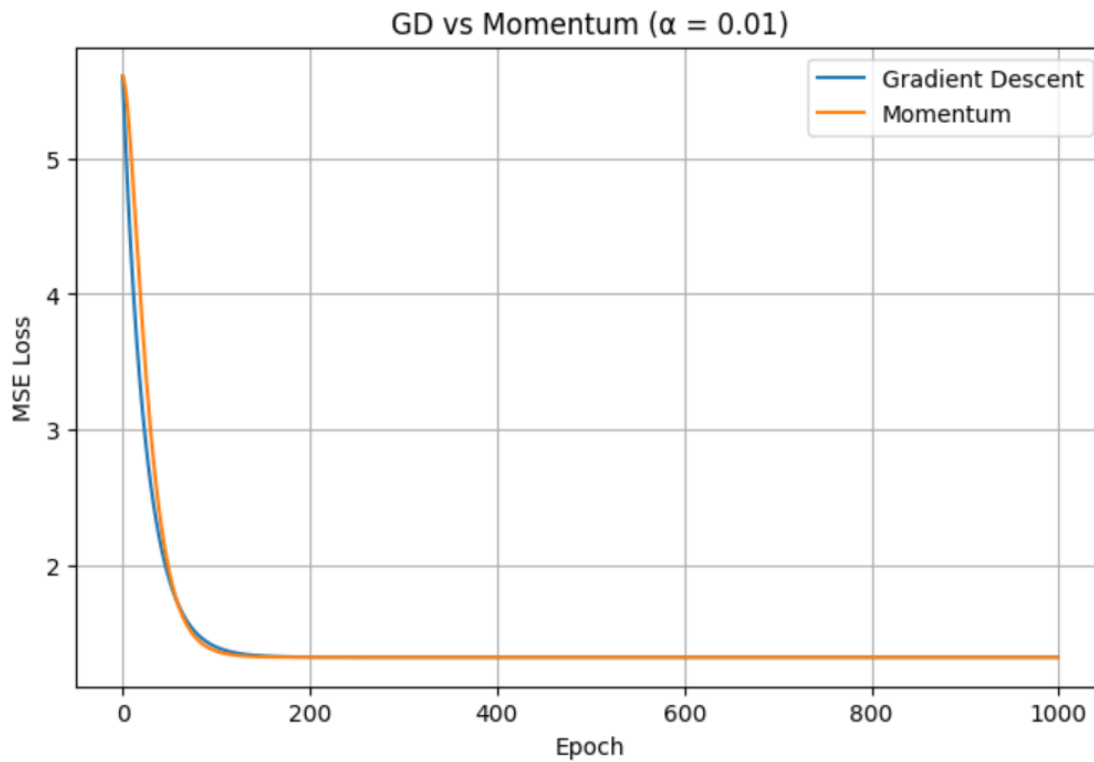


Figure 3: Gradient Descent versus Momentum at learning rate 0.01.

6.3 Comparison of the Runs

The two optimizers behave very similarly at learning rate 0.01.

Their final training MSE values differ only in the sixth decimal place.

At learning rate 0.001, both methods finish with a somewhat larger training MSE than their corresponding 0.01 runs.

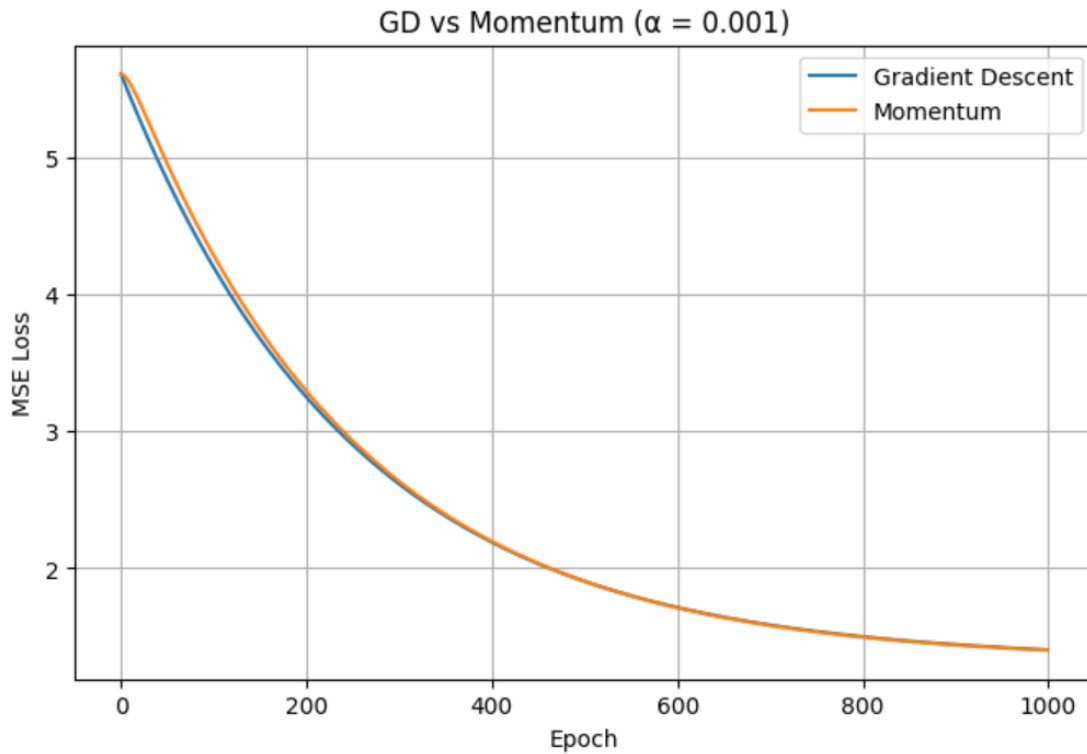


Figure 4: Gradient Descent versus Momentum at learning rate 0.001.

7. Testing

The test set containing 4,128 observations, but it does not contain a final test-prediction/MSE calculation.

Therefore, a numerical testing MSE should not be inserted into this report without running an additional evaluation cell.

8. Discussion

The experiment shows the complete learning cycle of a neural network.

Normalized inputs are propagated forward, an MSE is calculated, gradients are obtained by backpropagation, and the parameters are updated.

Repeating this process for 1000 epochs produces the convergence curves shown above.

The learning rate has a visible effect on the final training loss.

In these runs, the 0.01 learning rate gives a lower final training MSE than 0.001 for both optimization methods.

Momentum is marginally better than basic Gradient Descent at 0.01 in the recorded results.

It is important to note that only two input features are used.

Consequently, the experiment is mainly useful for understanding the mechanics of a neural network rather than for obtaining the best possible California Housing prediction.

9. Conclusion

A multi-layer neural network was implemented from scratch for predicting `MedHouseVal` from `MedInc` and `HouseAge`.

The model uses two ReLU hidden layers with five and three neurons, followed by a single linear output neuron.

Forward propagation, MSE loss, backpropagation, Gradient Descent, and Momentum were implemented directly with NumPy.

Across the four experiments, the lowest recorded training MSE was approximately

1.327936684

using Momentum with learning rate 0.01.

Overall, the implementation provides a straightforward demonstration of how a neural network learns through repeated prediction, error calculation, gradient computation, and parameter updates.

10. Key Experimental Values

Table 5: Summary of Experimental Values

Item	Value
Dataset	California Housing
Input features	MedInc, HouseAge
Target	MedHouseVal
Architecture	$2 \rightarrow 5 \rightarrow 3 \rightarrow 1$
Activation	ReLU in hidden layers; Linear output
Loss function	MSE
Learning rates	0.01, 0.001
Epochs	1000
Methods	Gradient Descent, Momentum
Best training configuration	Momentum, learning rate 0.01
Best final training MSE	1.327936684